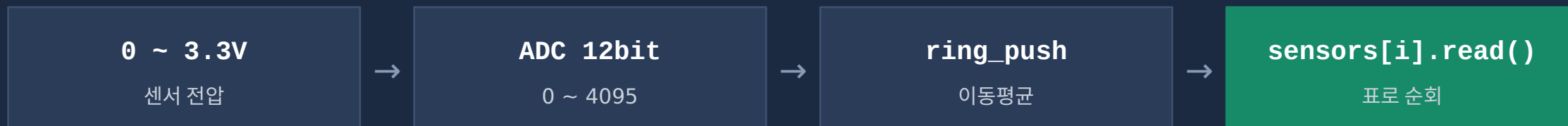


아날로그 입력 · 센서 · 코드 모듈화

전압을 숫자로 읽고, 흔들림을 걸러 내고, 센서를 구조체 표로 다룹니다



오늘의 8 교시

오전은 ADC · 필터 · 서보 · 온습도, 오후는 경보 · 거리 · 모듈화 · 종합 — 오늘 함정은 대부분 " 값이 이상하다 " 입니다

1

ADC 기초

가변저항, 12 비트, 평균 함수

2

조도 · 필터

분압, 링 버퍼 이동평균

3

조이스틱 · 서보

50Hz 펄스, my_map, 데드존

4

온도 · 습도

LM35 mV, DHT11 NaN, 구조체

5

가스 · 불꽃 · 소리

5V 분압, 히스테리시스

6

초음파 · PIR

$58\mu\text{s} = 1\text{cm}$, 타임아웃

7

수분 · 모듈화

젤 때만 전원, .h / .cpp, 링크

8

다중 센서 모니터

함수 포인터 표, CSV · 플로터

Day 1 은 0 과 1, Day 2 는 그 사이

버튼은 눌림 / 안 눌림 두 가지뿐이지만 , 센서는 연속된 전압을 냅니다 — ADC 가 그 전압을 숫자로 바꿉니다

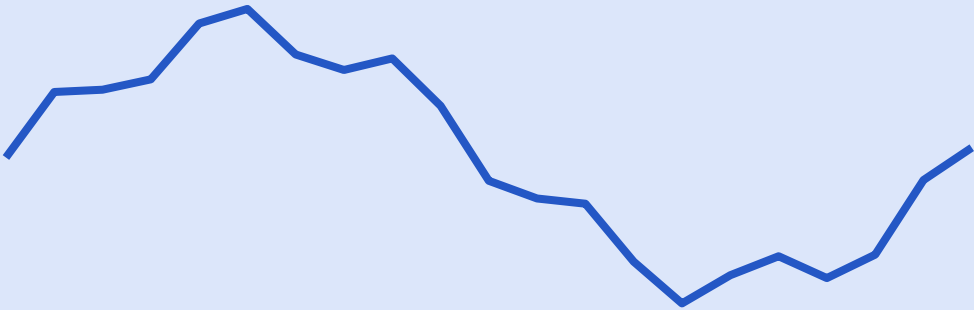
디지털 입력 (Day 1)



digitalRead → HIGH(1) / LOW(0)

버튼 · PIR · DHT11(데이터 선)

아날로그 입력 (Day 2)



analogRead → 0 ~ 4095 (12 비트)

가변저항 · 조도 · 조이스틱 · LM35 · MQ-2 · 토양 · 수위

	오늘 새로 쓰는 c	어디서
포인터 인자로 여러 값 돌려주기	int read_avg(int pin, int n, int *mn, int *mx)	1 교시
링 버퍼 (배열 + % 연산)	r->head = (r->head + 1) % RN	2 교시
구조체 반환 · NaN 검사	reading_t r = read_dht(); isnan(r.temp_c)	4 교시
헤더 분리 · extern	sensor.h / sensor.cpp	7 교시
함수 포인터 표	sensors[i].read()	8 교시

오늘의 전원 레일 — 3V3 · GND · 5V

Day 1 은 GND 하나였지만, 오늘은 5V로 동작하는 부품이 있어 레일을 세 가지로 씁니다



위 빨강 (+)

ESP32 3V3

가변저항 · 조도 · 조이스틱 · DHT11 · 불꽃 · 소리



위 · 아래 파랑 (-)

ESP32 GND ×2

모든 부품 — 전원이 달라도 GND는 하나



아래 빨강 (+)

ESP32 5V (필요할 때만)

서보 · LM35 · MQ-2 · HC-SR04 · PIR

5V 출력 신호는 반드시 분압

MQ-2 AO (최대 5V): $10k : 10k \rightarrow \div 2 = 2.5V$

HC-SR04 Echo (5V): $10k : 20k \rightarrow 3.33V$

LM35 Vout 은 $10mV/^{\circ}C$ 라 작아서 그대로

$$V_{out} = V_{in} \times R2 \div (R1 + R2)$$

R1 은 신호 쪽, R2 는 GND 쪽 — 분압점 (가운데) 을 GPIO 에

10kΩ 색띠 : 갈색 검정 주황

ADC1 여섯 핀을 교시마다 나눠 쓴다

Wi-Fi 와 함께 쓸 수 있는 ADC1: 32 · 33 · 34 · 35 · 36(VP) · 39(VN) — 34~39 는 입력 전용이라 센서에 딱 맞다

GPI0	1 교시	2 교시	3 교시	4 교시	5 교시	6 교시	7 교시	8 교시
34	가변저항				불꽃		토양	가변저항
35		조도			소리		수위	조도
32					MQ-2			
33				LM35				LM35
36 · 39			조이스틱 X · Y					
디지털		LED 25	SW 4 · 서보 18	DHT11 27	부저 19	Trig 17 · Echo 16 · PIR 13 · LED 25	전원 14 · 23	DHT11 27

교시마다 그 교시의 배선도가 있습니다 . 앞 교시 부품은 빼도 되고 , 열이 겹치지 않으면 그대로 뒤도 됩니다 . 34 · 35 는 교시마다 M-F 선만 옮겨 꽂습니다 .

센서 모듈의 핀 순서는 제품마다 다릅니다 — 기판의 인쇄 글자 (S / V / G, AO / VCC / GND) 를 읽고 그림의 글자와 맞춥니다 .

따라하기 · DHT 라이브러리 설치 (4 교시 전)

오늘 라이브러리가 필요한 센서는 DHT11 하나뿐 — 나머지는 직접 구현합니다

- 1 라이브러리 매니저** 왼쪽 세 번째 아이콘 (책) · 또는 스케치 → 라이브러리 포함하기 → 라이브러리 관리
- 2 검색** `DHT sensor library`
- 3 설치** `DHT sensor library by Adafruit` → 설치
- 4 모두 설치** " 필요한 라이브러리도 ? " → 모두 설치 (Adafruit Unified Sensor 함께)
- 5 확인** 출력 창 : `Installed DHT sensor library`

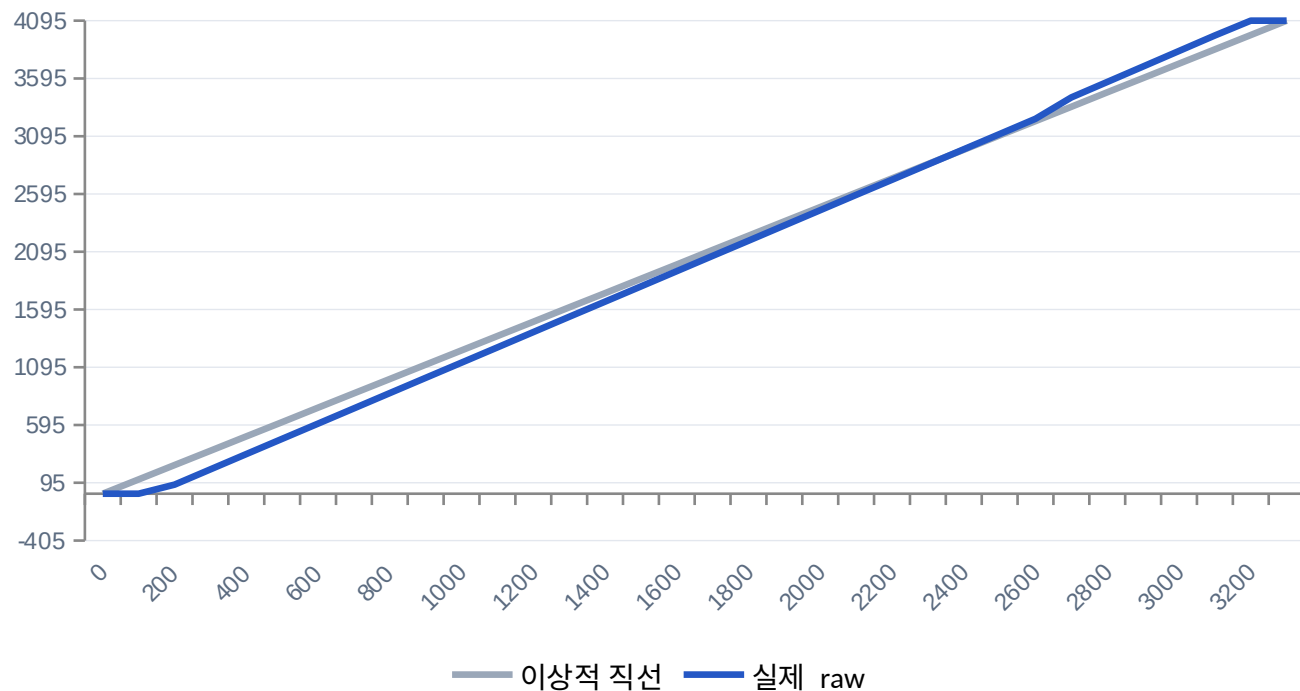
오류	조치
<code>DHT.h: No such file or directory</code>	3 번 누락 또는 다른 이름의 라이브러리 → Adafruit 것으로
<code>Adafruit_Sensor.h: No such file</code>	4 번에서 " 모두 설치 " 를 안 누름 → Adafruit Unified Sensor 따로 설치

ADC — 전압을 0~4095 로

1 교시

12 비트 = 4096 단계, 한 단계 $\approx 0.8\text{mV}$ · 기본 감쇠 11dB 로 약 0.15 ~ 3.1V 를 잰다

입력 전압 (mV) → analogRead



함수	돌려주는 것
<code>analogRead(34)</code>	0 ~ 4095 (칩마다 다름 , 양 끝 비선형)
<code>analogReadMilliVolts(34)</code>	칩 보정값 (eFuse) 으로 계산한 mV

0.15V 아래는 모두 0, 3.1V 위는 4095. 전압이 필요하다면 `analogReadMilliVolts`.

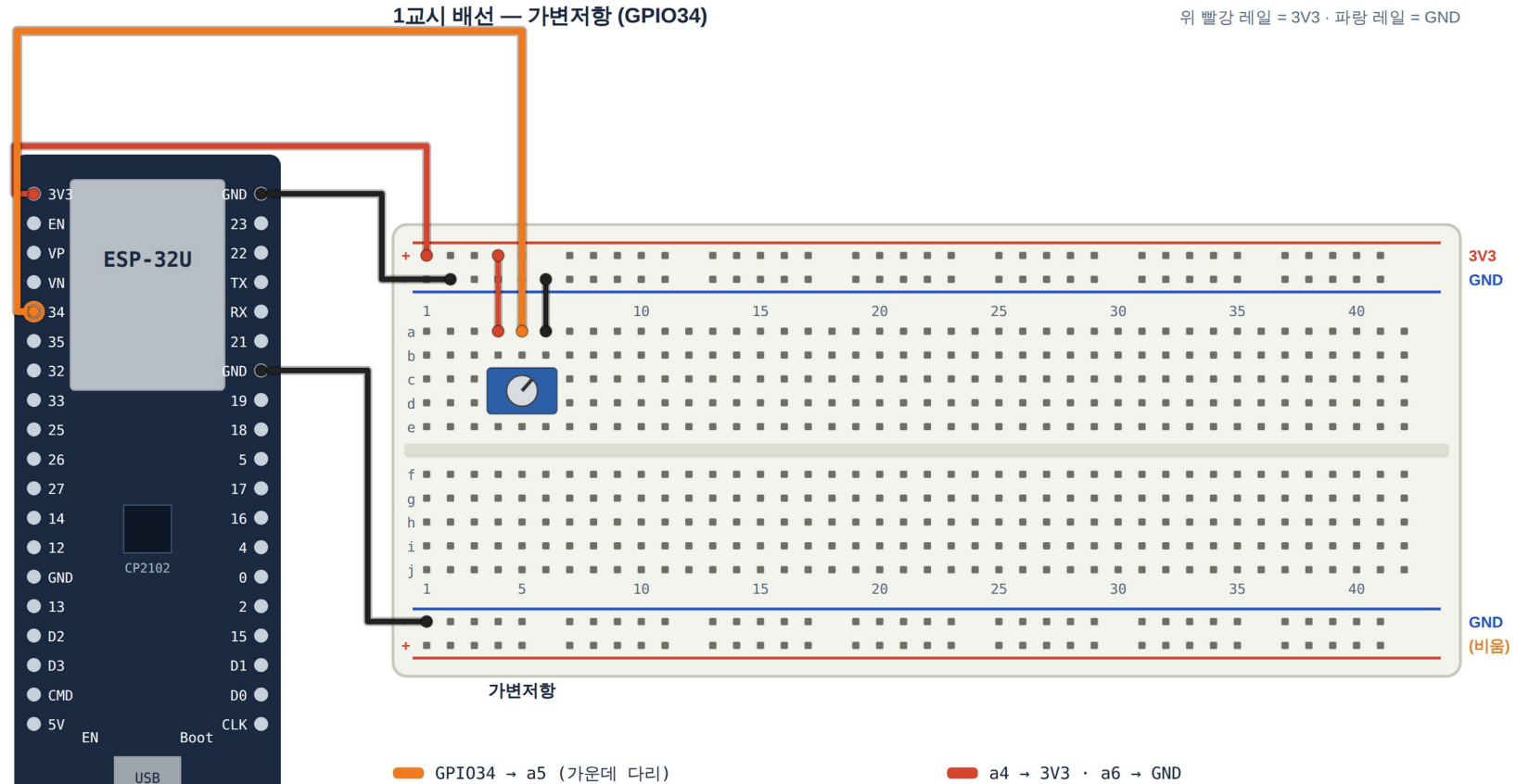
⚠ ADC2(0 · 2 · 4 · 12~15 · 25~27) 는 Wi-Fi 를 켜면 읽을 수 없다 → 센서는 처음부터 ADC1(32~39) 에

🎬 D2-1 ADC 변환기 — 전압 슬라이더 , ADC2 + Wi-Fi 토글

1 교시 배선 — 가변저항

1 교시

양 끝 다리는 3V3 · GND, 가운데 다리 (와이퍼) 만 GPIO34 — 끝 다리를 바꾸면 방향만 반대



가변저항 한쪽을 5V 에 연결하면 GPIO 에 3.3V 를 넘는 전압이 들어갈 수 있습니다 . 아날로그 센서 전원은 3V3.

여러 번 읽어 평균 — 결과가 여럿이면 포인터로

1 교시

반환값은 하나 — 최소 · 최대는 호출자의 변수 주소에 써 준다 (C 과정 Day 2 max_to 모양)

```
int read_avg(int pin, int n, int *min_out, int *max_out) {
    long sum = 0;
    int mn = 4095, mx = 0;
    for (int i = 0; i < n; i++) {
        int v = analogRead(pin);
        sum += v;
        if (v < mn) mn = v;
        if (v > mx) mx = v;
    }
    *min_out = mn;
    *max_out = mx;
    return sum / n;
}

int mn, mx;
int avg = read_avg(34, 32, &mn, &mx);
```

```
avg=2040  min=2021  max=2063  noise=42
avg=2041  min=2019  max=2066  noise=47
```

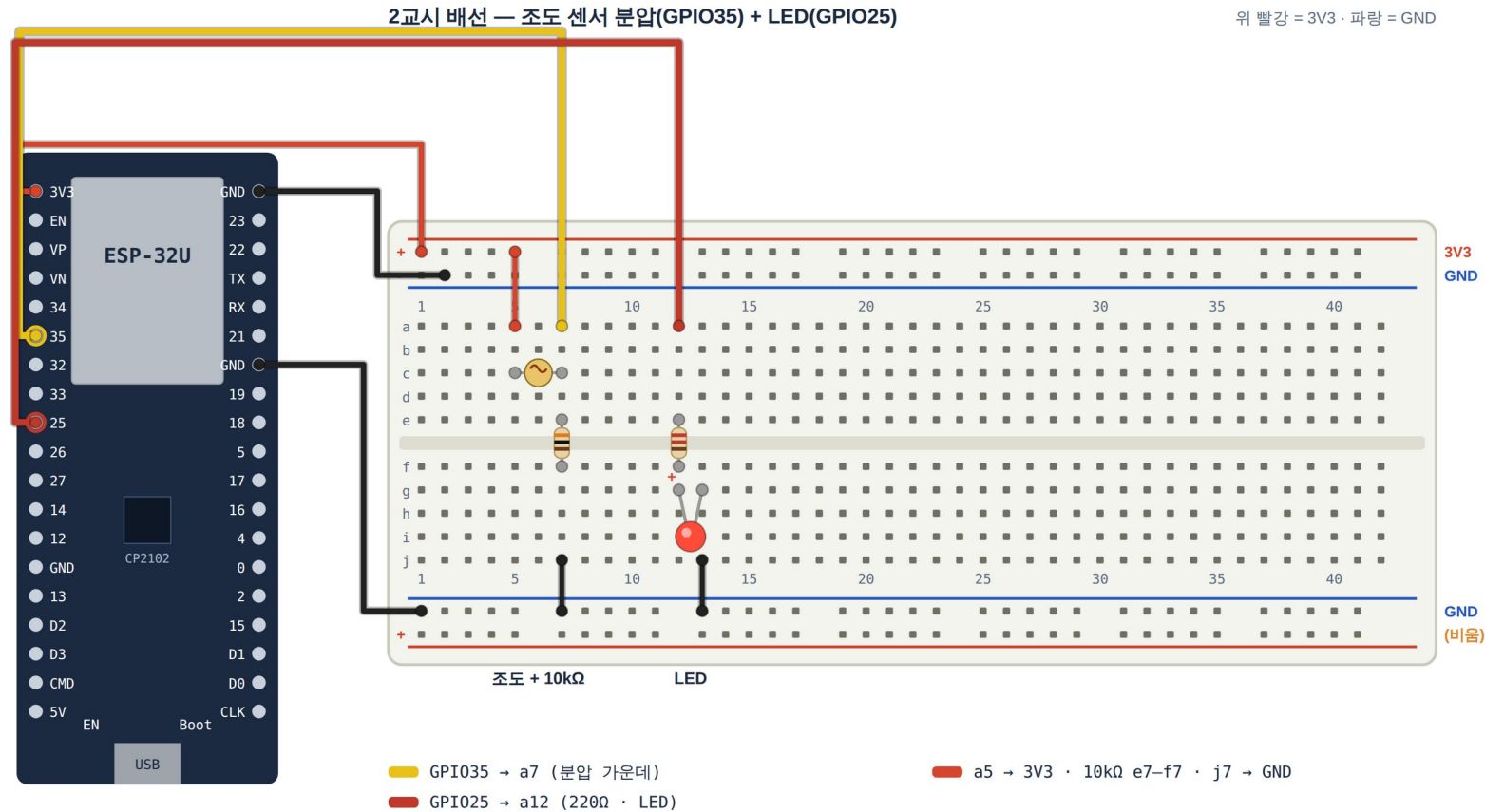
손을 대지 않아도 noise 가 수십 — 정상입니다 . 2 교시에 링 버퍼 이동평균으로 줄입니다 .

pinMode 는 필요 없다 . analogRead 가 핀을 ADC 로 설정한다 . 풀업을 켜면 전압이 들어진다 .

2 교시 배선 — 조도 센서 분압 + LED

2 교시

3V3 → 포토레지스터 → (GPIO35) → 10kΩ → GND · LED 는 Day 1 과 같은 모양 (GPIO25)

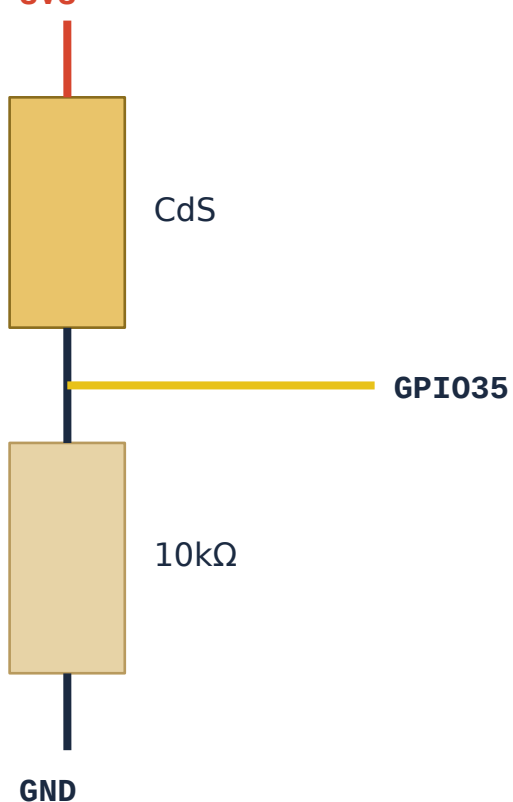


LED 를 조도 센서 바로 옆에 두면 LED 빛이 센서로 들어가 깜빡이며 진동합니다 — 떨어뜨리거나 가립니다 .

저항을 전압으로 — 전압 분배

2 교시

ADC 는 저항을 직접 못 잴다 → 10kΩ 과 직렬로 연결해 가운데 전압을 읽는다



$$V_{out} = 3.3V \times 10k \div (R_{CdS} + 10k)$$

상황	R_CdS	Vout	raw
밝음 (손전등)	1kΩ	3.0V	약 3700
실내	10kΩ	1.65V	약 2000
손으로 가림	100kΩ	0.3V	약 300

실수	결과
10kΩ 없이 CdS 만	밝기와 상관없이 4095 근처
10kΩ 대신 220Ω	항상 작은 값 — 색띠 확인

링 버퍼 이동평균 — 흔들림 줄이기

2 교시

최근 RN 개의 평균 · 합계를 들고 다니면 RN 이 커져도 더하기 한 번 · 빼기 한 번

```
#define RN 8
typedef struct {
    int buf[RN];    // 최근 RN 개
    int head;       // 다음에 쓸 위치
    int count;      // 들어온 개수 ( 최대 RN )
    long sum;       // buf 합계
} ring_t;

int ring_push(ring_t *r, int v) {
    if (r->count == RN) r->sum -= r->buf[r->head];
    else r->count++;
    r->buf[r->head] = v;
    r->sum += v;
    r->head = (r->head + 1) % RN;
    return r->sum / r->count;
}
```



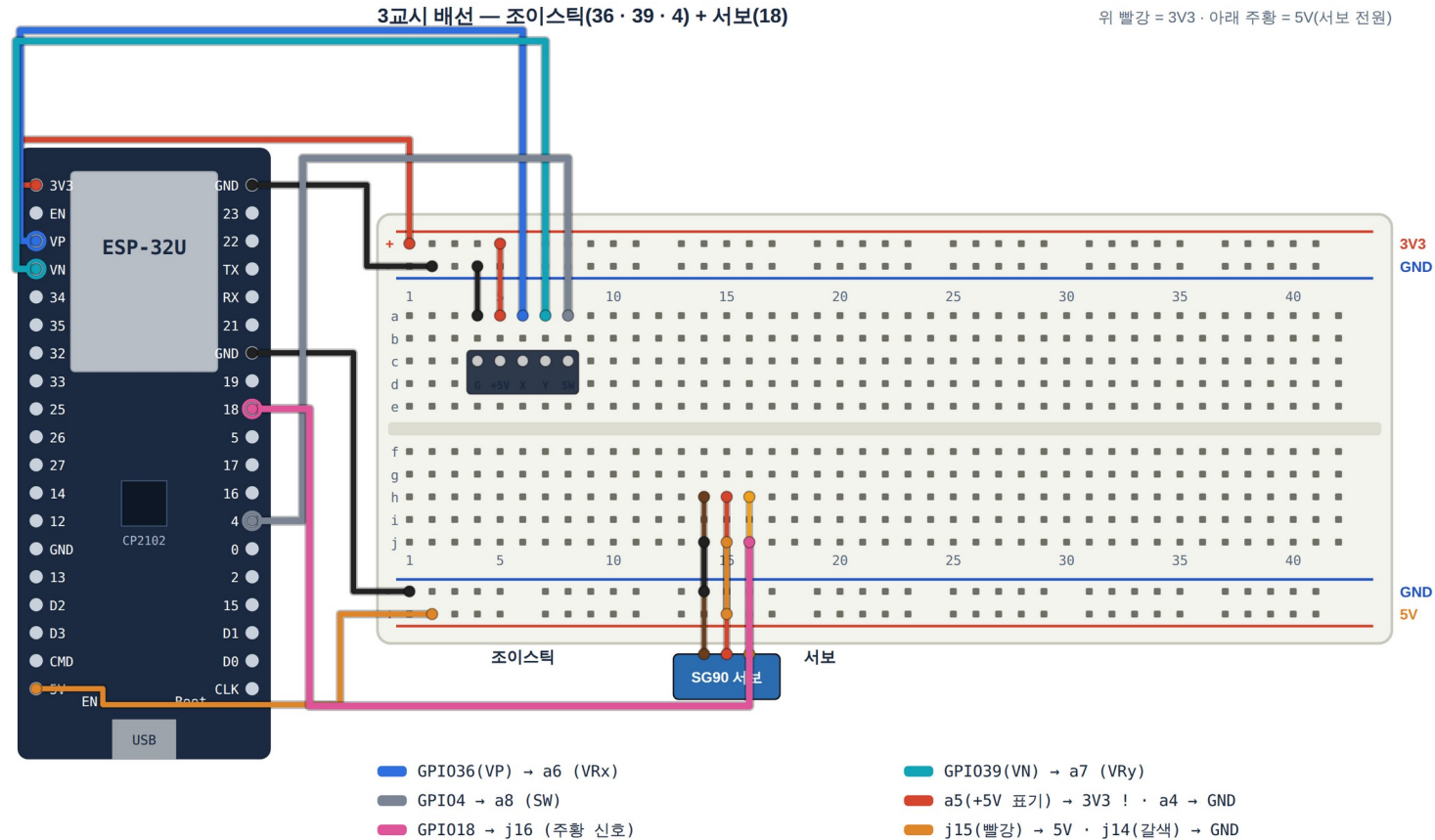
head 가 끝에 닿으면 % RN 으로 0 으로 → 가장 오래된 값을 덮어쓴다. 가득 차기 전에는 count 로 나눈다.

🎬 D2-2 · 한 번 push / 재생 — 링 버퍼 칸이 채워지는 과정

3 교시 배선 — 조이스틱 + 서보

3 교시

조이스틱은 3V3 (인쇄가 +5V 여도 !) · 서보는 5V · 서보 GND 는 ESP32 GND 와 같은 레일

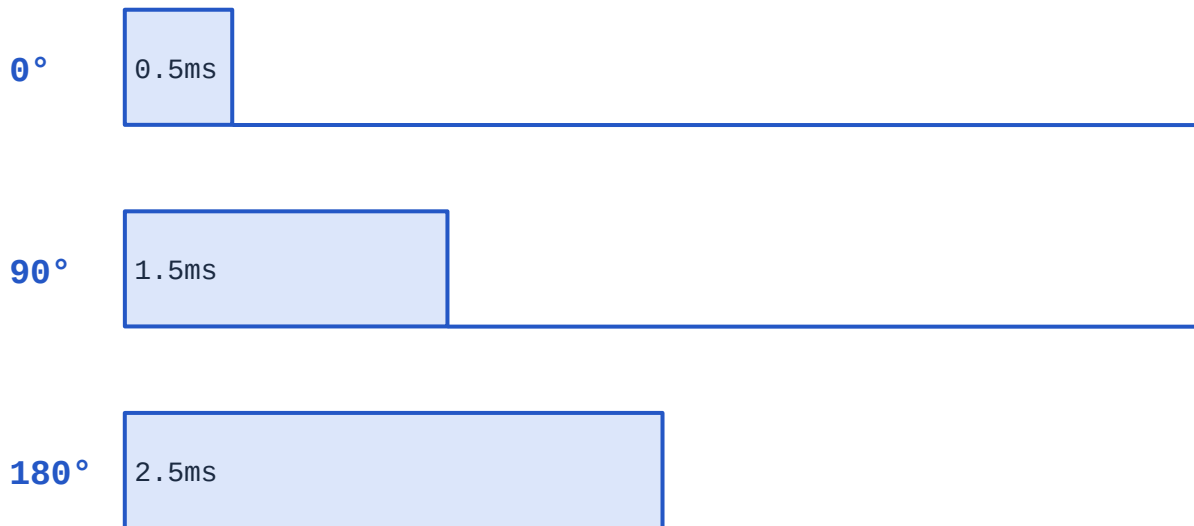


조이스틱에 5V 를 주면 X · Y 출력이 3.3V 를 넘어 ADC 가 4095 에 붙고 핀이 위험합니다 .

서보 — 20ms 중 HIGH 시간이 각도

3 교시

Day 1 의 LEDC 로 50Hz 를 만들면 라이브러리 없이 서보를 돌릴 수 있다



← 20ms (50Hz) 중 앞부분만 확대 →

```
const int PIN_SERVO = 18;
const int BITS = 14;           // 0 ~ 16383
const int MAX_DUTY = (1 << BITS) - 1;

void servo_write_us(int us) {
    ledcWrite(PIN_SERVO, (long)us * MAX_DUTY / 20000);
}
void servo_write_deg(int deg) {
    servo_write_us(500 + deg * 2000 / 180);
}

ledcAttach(PIN_SERVO, 50, BITS); // 50Hz
```

14 비트면 한 단계 $\approx 1.2\mu\text{s}$ 로 부드럽고, 8 비트면 $78\mu\text{s}$ 라 거칠다. 5000Hz 로 설정하면 서보가 펄스를 해석하지 못한다.

🎬 D2-3 조이스틱 → 서보 — 펄스 폭 그림

my_map 과 데드존

3 교시

비례식은 곱하기 먼저 , 나누기 나중 · 가운데 근처 잡음은 무시

```
long my_map(long x, long in_min, long in_max,
            long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min)
        / (in_max - in_min) + out_min;
}

int center = analogRead(36);    // setup 에서 한 번

int x = analogRead(36);
int deg = (abs(x - center) < 150) ? 90
        : my_map(x, 0, 4095, 0, 180);
servo_write_deg(deg);
```

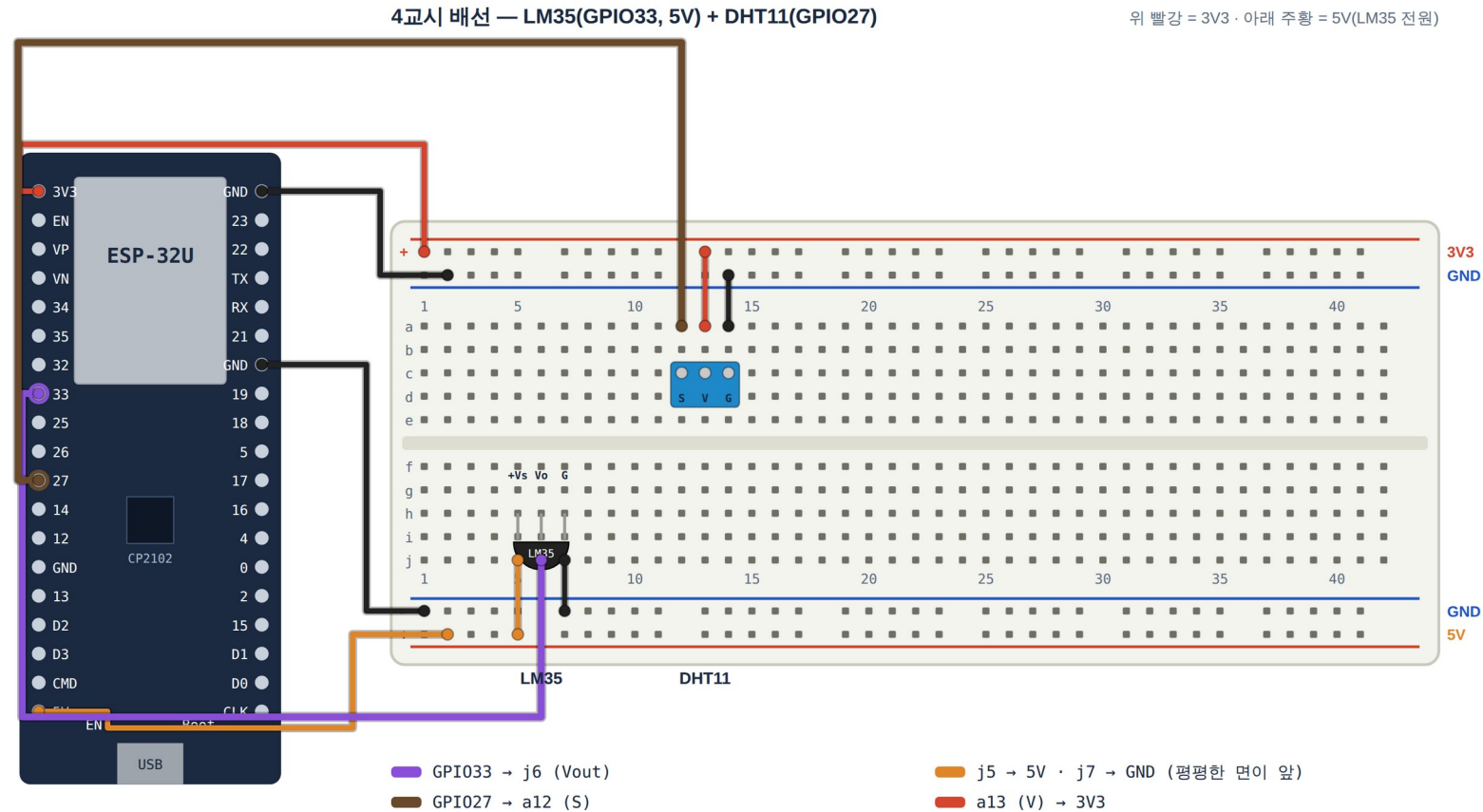
실수	결과
서보 빨강 → 3V3	떨림 · ESP32 리셋 (brown-out)
서보 GND 공통 안 함	무작위로 떨림
ledcAttach(18, 5000, 8)	서보가 반응 안 함 → 50Hz
$2048 / 4095 * 180$	정수 나눗셈 → 0 — 곱하기 먼저
데드존 없음	가운데에서도 움찔

데드존은 " 이 정도 흔들림은 무시 " — 5 교시 히스테리시스와 같은 발상입니다 .

4 교시 배선 — LM35 + DHT11

4 교시

LM35: 평평한 면이 앞, 왼쪽부터 +Vs(5V) · Vout(GPIO33) · GND · DHT11: S(27) · V(3V3) · G



LM35 를 거꾸로 꽂으면 뜨거워집니다 — 만져서 뜨거우면 즉시 USB 분리 . LM35 는 4V 이상 필요 → 5V 레일 .

LM35 는 전압으로 , DHT11 은 40 비트로

4 교시

아날로그 센서는 mV 를 계산 , 디지털 센서는 라이브러리가 해석 — 실패 (NaN) 는 반드시 검사

10mV = 1°C

245mV → 24.5°C · 25°C 에 0.25V 뿐 → 분압 불필요
ADC 하한 근처라 analogReadMilliVolts + 16 회 평균

센서	정확도
LM35	±0.5°C (25°C)
DHT11	±2°C · 분해능 1°C

48

습도 정수

0

습도 소수

25

온도 정수

0

온도 소수

73

체크섬

48 + 0 + 25 + 0 = 73 → 체크섬 일치 · 8 비트 × 5 = 40 비트 , 선 하나로

```
#include <DHT.h>
DHT dht(27, DHT11);

float t = dht.readTemperature();
if (isnan(t)) {           // t == NAN 은 항상 거짓 !
    Serial.println("DHT 읽기 실패");
}
delay(2000);              // 1~2 초 간격
```

측정값 구조체 — 값 + 언제 + 유효한가

구조체는 값으로 통째로 반환할 수 있다 (C 과정 Day 3) — 8 교시에 배열로 확장

```
typedef struct {
    float temp_c;
    float humi;          // 습도 없는 센서는 NAN
    uint32_t t_ms;       // 잦은 시각
    bool ok;
} reading_t;

reading_t read_dht(void) {
    reading_t r;
    r.temp_c = dht.readTemperature();
    r.humi    = dht.readHumidity();
    r.t_ms    = millis();
    r.ok = !isnan(r.temp_c) && !isnan(r.humi);
    return r;
}
```

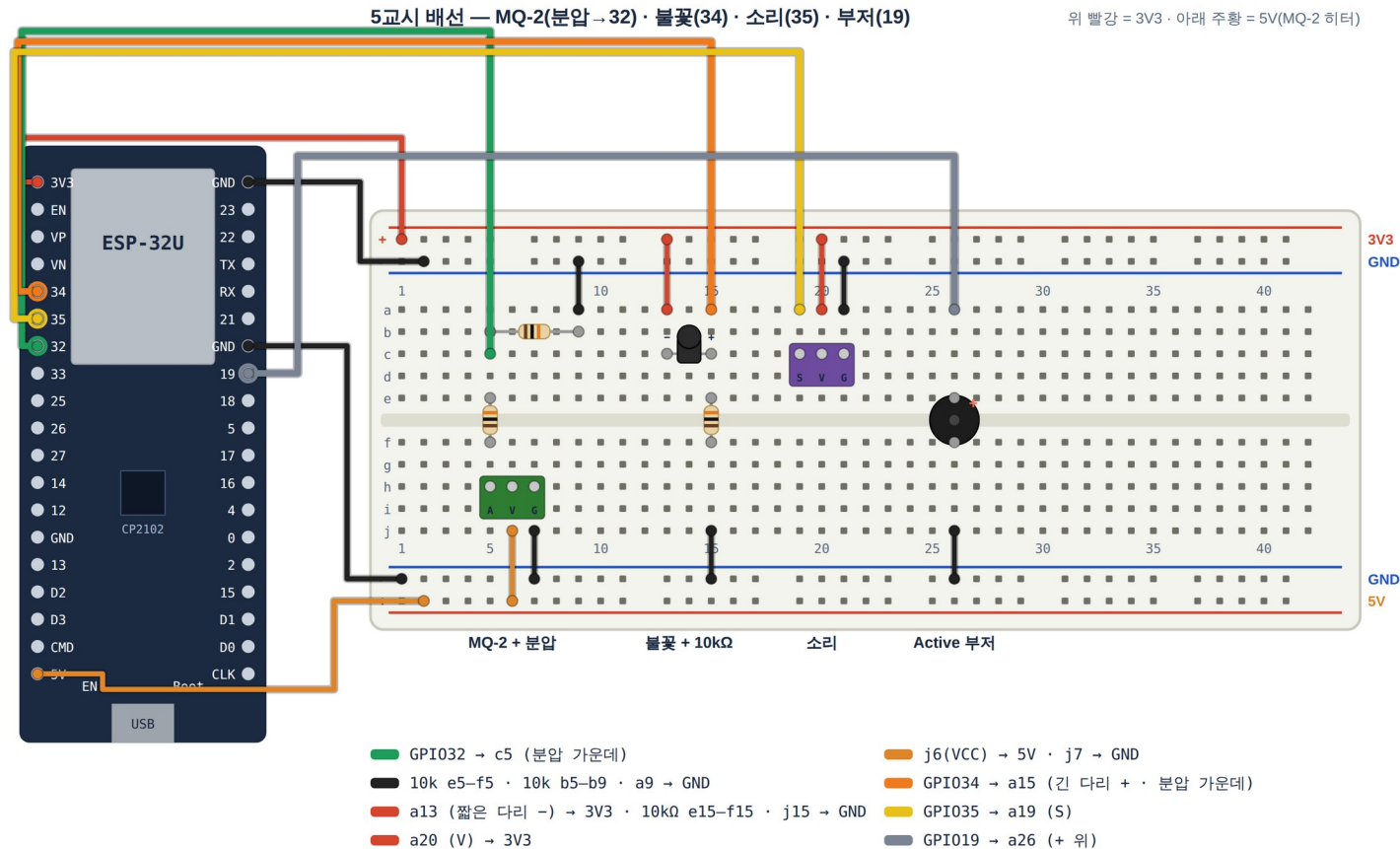
실수	결과
LM35 거꾸로	뜨거워짐 → 즉시 USB 분리
LM35 를 3V3 에	값 불안정 · 0
DHT11 을 100ms 마다	절반 이상 NaN
if (t == NAN)	항상 거짓 → isnan(t)
라이브러리 한쪽만 설치	Adafruit_Sensor.h 오류

LM35 와 DHT11 이 1~2°C 다른 것은 정상 — 두 센서의 정확도 차이입니다 .

5 교시 배선 — MQ-2 · 불꽃 · 소리 · 부저

5 교시

MQ-2 AO → 10k : 10k 분압 → 32 · 불꽃 (2 핀) 긴 다리 → 34 + 10kΩ, 짧은 다리 → 3V3 · 소리 S → 35 · 부저 → 19



MQ-2 는 전원을 넣고 1~2 분 예열 · 모듈이 따뜻해지는 것은 정상 · 테스트는ライター 가스를 불 없이 살짝 .

5V 출력은 반으로 — MQ-2 와 분압

5 교시

AO 는 가스가 많을수록 올라가 5V 까지 → 절반으로 만들어 GPIO 에

$$V_{gpio} = V_{AO} \times 10k \div (10k + 10k) = V_{AO} \div 2$$

```
const int PIN_GAS = 32;

int mv = analogReadMilliVolts(PIN_GAS);
int ao_mv = mv * 2;          // 분압 전 실제 AO
Serial.printf("gas_mv:%d\n", ao_mv);
```

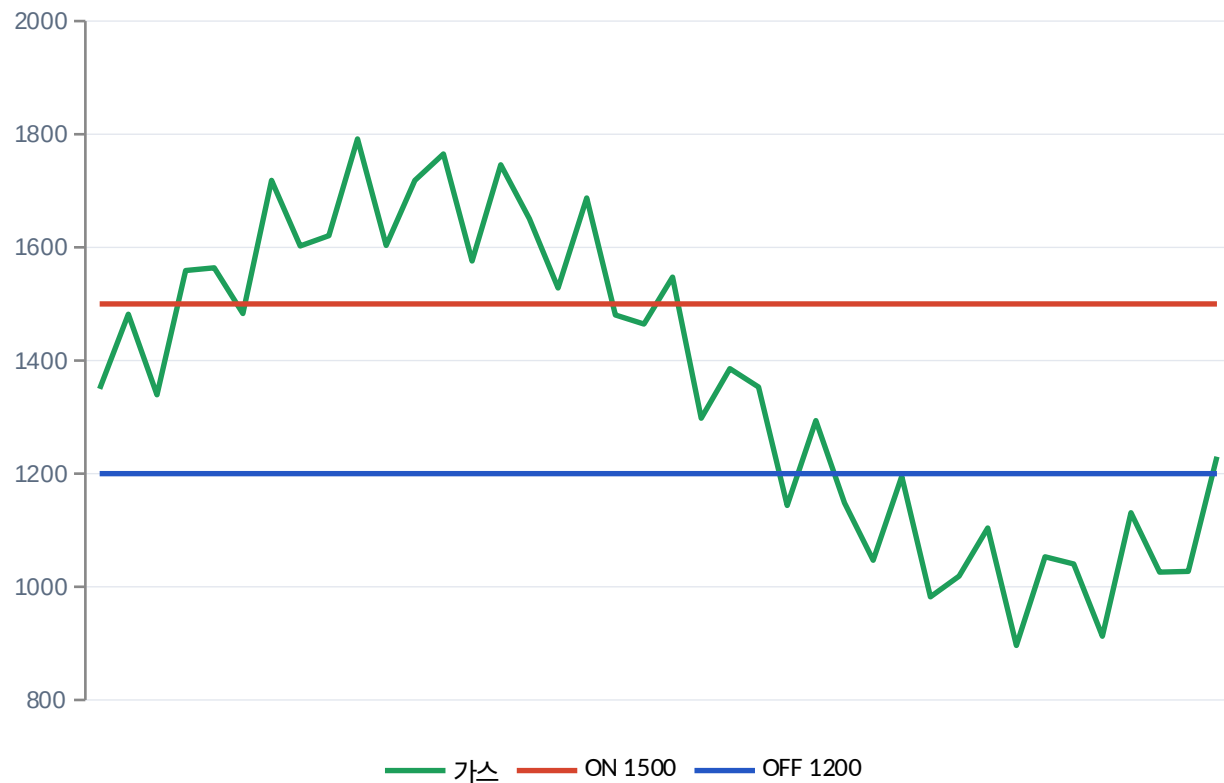
```
int sound_max = 0;
for (int i = 0; i < 200; i++) {
    int v = analogRead(35);
    if (v > sound_max) sound_max = v;
} // 소리는 파동 → 짧은 시간의 최대값
```

센서	확인할 것
MQ-2	5V 전원 · 분압 · 1~2 분 예열 뒤 깨끗한 공기 값 보고 임계값
불꽃 (2 핀)	짧은 다리 (-) → 3V3, 긴 다리 (+) → GPIO34 + 10kΩ → GND · 적외선이 많을수록 값 ↑ · 리모컨으로도 확인
소리	한 번 읽기 X → 최대값 (또는 최대 - 최소). 손뼉으로 확인
소리 모듈 핀	S V G · AO VCC GND · DO 까지 4 핀 — 아날로그 (S · AO) 를 GPIO 에

히스테리시스 — 정보가 떨어지지 않게

5 교시

켜는 선과 끄는 선을 다르게 두면, 그 사이에서는 이전 상태를 유지한다



임계값 하나면 1500 근처 잡음마다 켜졌다 꺼졌다 — 부저가 떨린다. 간격은 잡음보다 넓게.

🎬 D2-5 히스테리시스 — 임계값 1 개 / 2 개 비교

```
typedef struct {
    int pin, on_th, off_th;
    bool alarm;
} alarm_t;

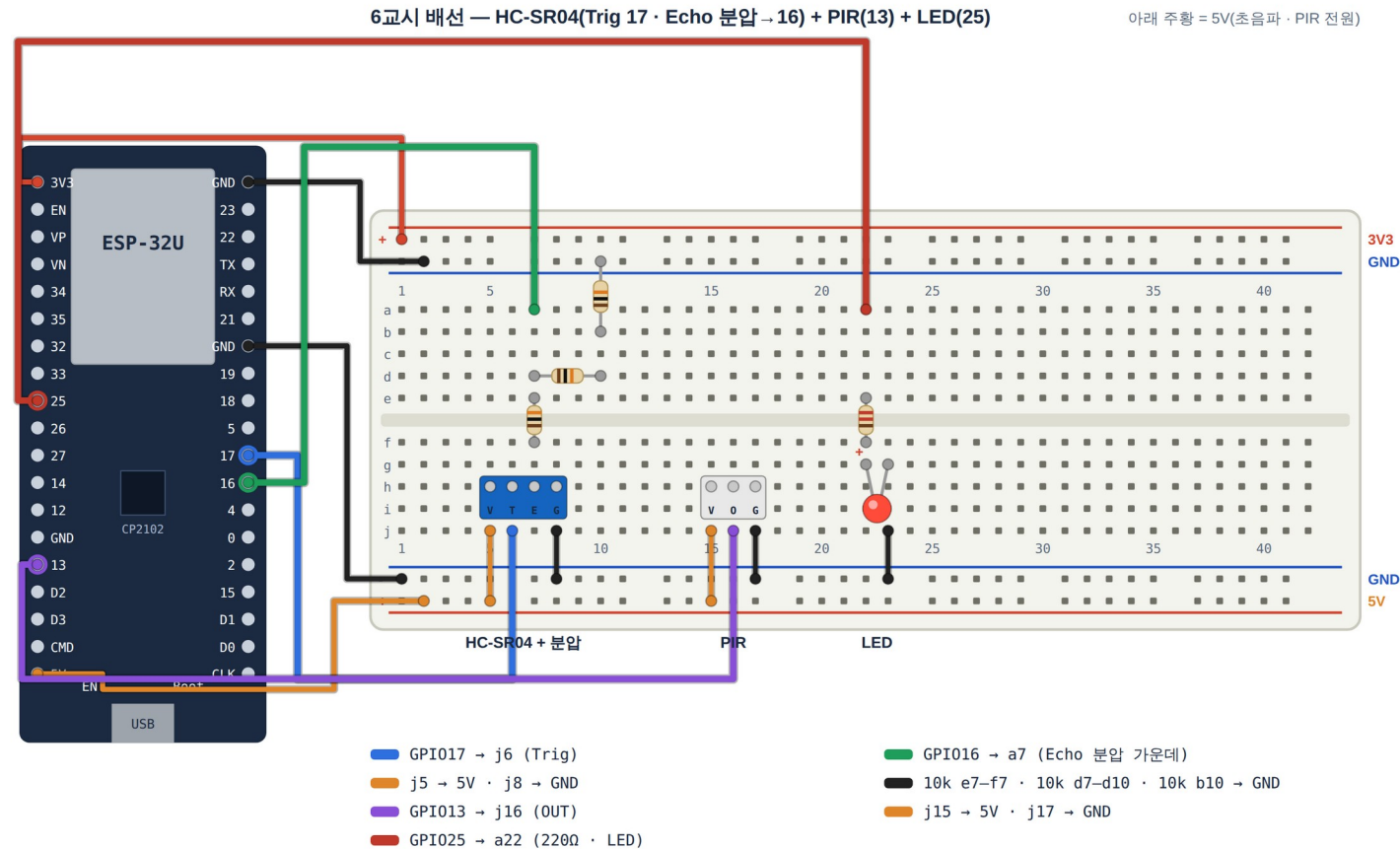
bool update_alarm(alarm_t *a, int v) {
    if (!a->alarm && v > a->on_th)
        a->alarm = true;
    else if (a->alarm && v < a->off_th)
        a->alarm = false;
    return a->alarm;
}

any |= update_alarm(&gas, gas_mv);
```

6 교시 배선 — HC-SR04 + PIR + LED

6 교시

HC-SR04: VCC 5V · Trig 17 · Echo → 10k : 20k 분압 → 16 · PIR: VCC 5V · OUT 13 · LED 25

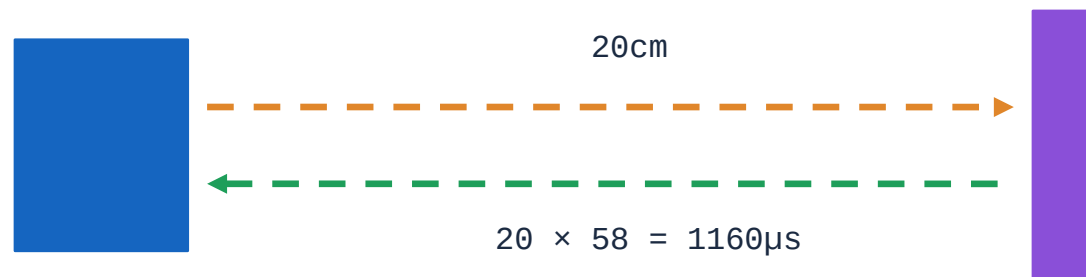


Echo 는 5V 펄스 — 분압 없이 연결하면 당장은 동작해 보여서 더 위험합니다 . 20kΩ = 10kΩ 두 개 직렬 .

소리가 갔다 오는 시간 = 거리

6 교시

소리 343m/s → 1cm 에 29μs, Echo 는 왕복이라 1cm 당 58μs



```
float read_cm(void) {  
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);  
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);  
    digitalWrite(PIN_TRIG, LOW);  
    unsigned long us = pulseIn(PIN_ECHO, HIGH, 30000);  
    if (us == 0) return -1; // 타임아웃 = 실패  
    return us / 58.0f;  
}
```

실수	결과
Echo 분압 없음	5V 가 GPIO 에 — 손상 위험
VCC 를 3V3 에	거리 짧아짐 · 항상 타임아웃
pulseIn 타임아웃 없음	물체 없으면 1 초씩 멈춤
us == 0 을 0cm 로	실패를 거리로 오해

30000μs ≈ 5m 왕복 . HC-SR04 측정 한계는 약 4m, 2cm 아래는 잘 못 잴다 .

🎬 D2-6 초음파 왕복 시간 — 거리 슬라이더 · 측정 버튼

PIR 움직임 + 초음파 거리 = 침입 감지

6 교시

PIR 은 넓게 "움직임이 있다", 초음파는 좁게 "얼마나 가까운가" — AND 로 오작동을 줄인다

```
bool moving = digitalRead(PIN_PIR);  
float cm = read_cm();  
bool near = (cm > 0 && cm < 30);  
  
digitalWrite(PIN_LED, moving && near);
```

두 초음파 센서를 동시에 쓰면 서로의 소리를 듣습니다 — 번갈아 측정합니다 .

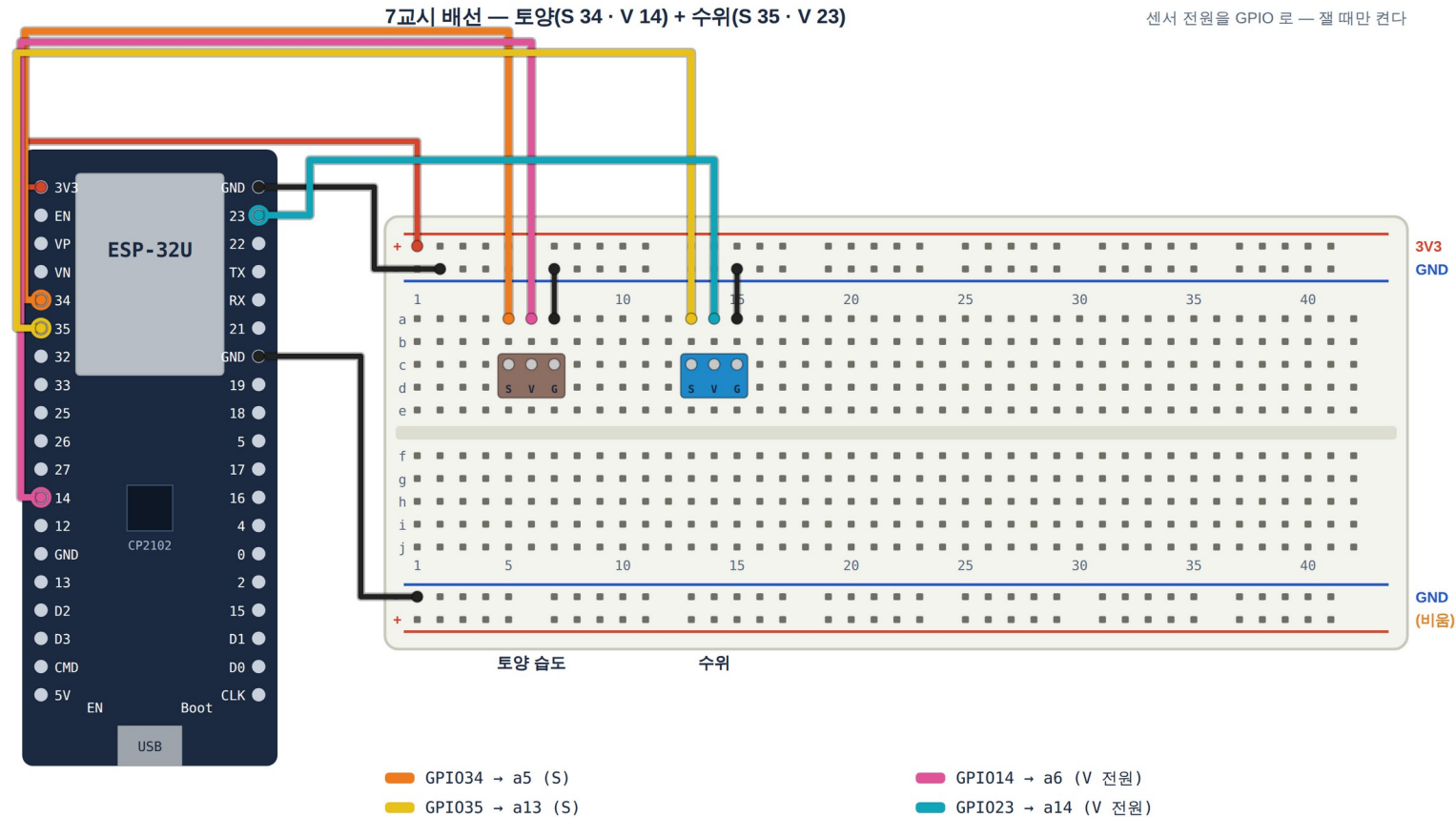
빈칸 6-1: 타임아웃이 3 번 " 연속 " 이면 " 센서 확인 " — 성공하면 fail_count = 0.

PIR (HC-SR501)	
감지	적외선 (체온) 의 변화 — 가만히 서 있으면 못 잡음
예열	전원 ON 후 30 초 ~ 1 분은 출력이 제멋대로
가변저항 2 개	감도 (거리) · HIGH 유지 시간
점퍼	반복 감지 모드
전원 · 출력	VCC 5V · OUT 은 3.3V → GPIO 에 그대로

7 교시 배선 — 토양 · 수위 (전원을 GPIO 로)

7 교시

토양 : S → 34 · V ← GPIO14 · 수위 : S → 35 · V ← GPIO23 — 젤 때만 켜다



전극에 전기가 계속 흐르면 전기 분해로 부식됩니다 . 물에는 전극 부분만 담그고 기판 윗부분은 젖지 않게 .

파일 나누기 — .h 는 약속 , .cpp 는 실체

7 교시

스케치 탭 ... → 새 탭 → sensor.h, sensor.cpp — 같은 스케치 폴더에 생긴다

```
// sensor.h
#ifndef SENSOR_H
#define SENSOR_H
typedef struct {
    int pin, pwr;
    const char *name;
} powered_sensor_t;
void sensor_init(const powered_sensor_t *s);
int sensor_read(const powered_sensor_t *s);
extern int sensor_read_count;
#endif
```

```
// sensor.cpp
#include <Arduino.h>
#include "sensor.h"
int sensor_read_count = 0;    // 정의는 한 곳

int sensor_read(const powered_sensor_t *s) {
    digitalWrite(s->pwr, HIGH); delay(10);
    int v = analogRead(s->pin);
    digitalWrite(s->pwr, LOW);
    sensor_read_count++;
    return v;
}
```

파일	담는 것	결과
sensor.h	타입 · 함수 원형 · extern 선언	#include 된 곳에 복사
sensor.cpp	함수 몸체 · 전역 변수 정의	sensor.cpp.o
water_soil.ino	setup · loop · sensor_read(&soil)	water_soil.ino.cpp.o → 링크 → .elf

링크 오류 읽기

오류에 ld 또는 collect2 가 보이면 컴파일러가 아니라 링커가 낸 것

한 일	오류	원인 · 해결
sensor.h 에 int sensor_read_count = 0;	multiple definition of 'sensor_read_count'	헤더를 포함한 파일마다 정의 → 헤더에는 extern 선언만
sensor.cpp 의 정의를 지움	undefined reference to 'sensor_read_count'	약속 (extern) 만 있고 실제 없음 → .cpp 에 정의 한 번
include guard 없이 두 번 포함	conflicting declaration / redefinition	같은 typedef 두 번 → #ifndef · #define · #endif
탭 이름을 sensor.c 로	undefined reference	.c 는 C, .ino 는 C++ 로 컴파일 → 이름 규칙 다름 → .cpp 로

C 과정 Day 1 의 전처리 → 컴파일 → 링크 그대로 . 자세한 출력을 켜 두면 빌드 로그에 sensor.cpp.o 가 따로 만들어지는 것이 보입니다 .

센서를 표로 — 함수 포인터 배열

8 교시

Day 1 8 교시 table[current]() 처럼, 센서마다 읽는 함수의 주소를 구조체에 넣는다

```
typedef struct {
    const char *name;
    float (*read)(void);    // 센서마다 다른 함수
    float value;
    uint32_t period_ms, last_ms;
} sensor_t;

sensor_t sensors[] = {
    {"pot",    read_pot,    0, 100,  0},
    {"light",  read_light,  0, 100,  0},
    {"lm35",   read_lm35,   0, 500,  0},
    {"dht_t",  read_dht_t,  0, 2000, 0},
};

const int NS = sizeof(sensors) / sizeof(sensors[0]);

for (int i = 0; i < NS; i++)
    if (now - sensors[i].last_ms >= sensors[i].period_ms) {
        sensors[i].value = sensors[i].read();
        sensors[i].last_ms = now;
    }
```

```
pot:2041.0, light:1733.0, lm35:24.6, dht_t:25.0
```

```
# m 을 보낸 뒤 (CSV)
ms,pot,light,lm35,dht_t
15230,2040.0,1731.0,24.6,25.0
15330,2043.0,1730.0,24.6,25.0
```

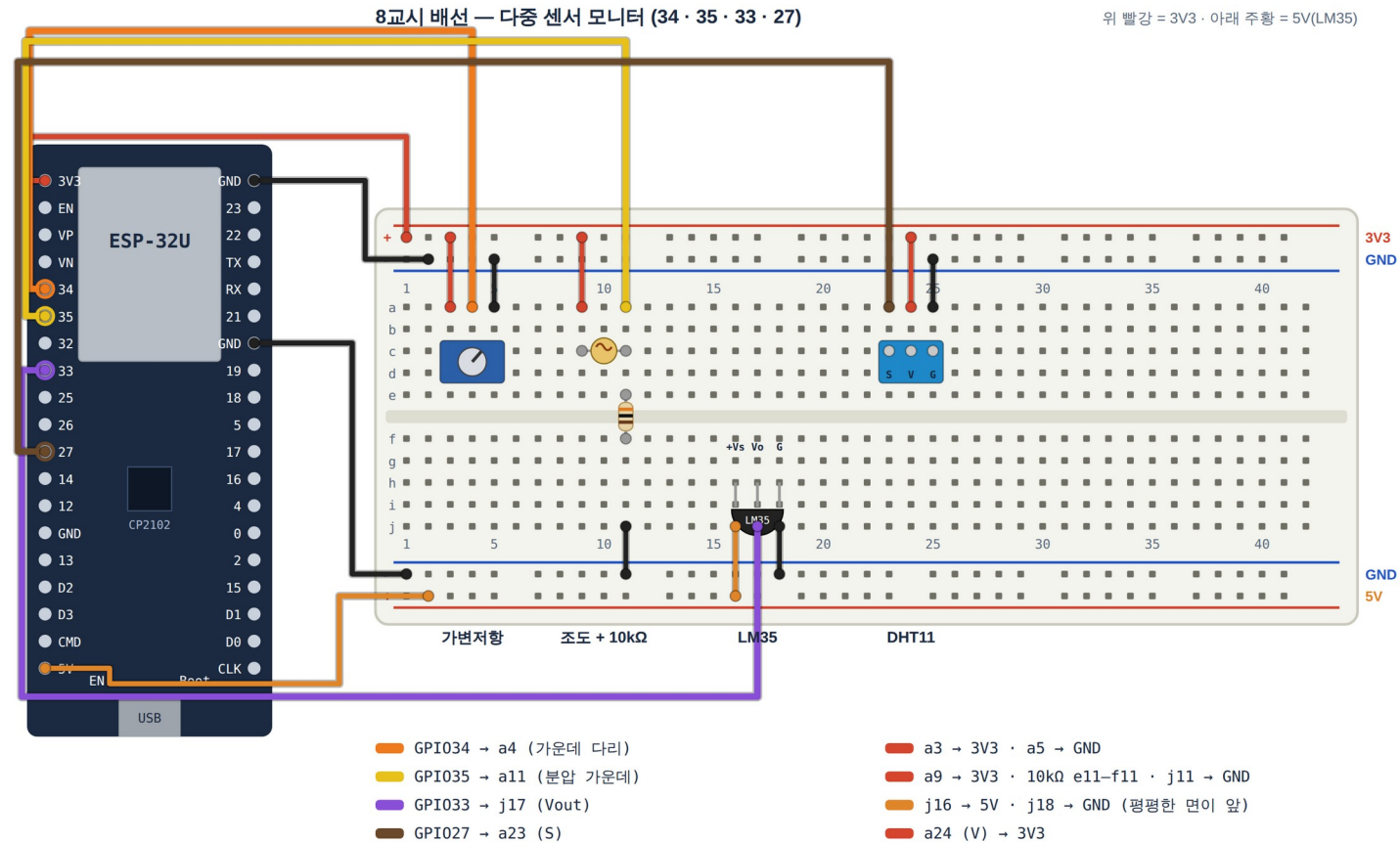
센서 추가 = 읽기 함수 하나 + 표에 한 줄 . 출력 · 주기 처리 코드는 그대로 .

DHT 는 2 초 , 가변저항은 0.1 초 — delay 로 전체를 늦추지 않는다 (Day 1 millis 비교).

 D2-8 CSV → 시리얼 플로터 — 재생 · m 보내기

8 교시 배선 — 다중 센서 모니터

1 · 2 · 4 교시 부품을 한 보드에 : 가변저항 (34) · 조도 (35) · LM35(33, 5V) · DHT11(27)



5V 레일은 LM35 전원에만 씁니다. 나머지 센서는 3V3 레일.

종합 과제 8-A · 다중 센서 모니터

8 교시

TODO 1~5 를 채운다 — 시리얼로 m 을 보내면 플로터 형식 ↔ CSV 형식

TODO	할 일
1	read_dht_t — NaN 이면 -99
2	DHT 온도를 2000ms 주기로 표에 추가
3	주기가 지난 센서만 read() 호출 , last_ms 갱신
4	CSV 머리줄 "ms,pot,light,lm35,dht_t"
5	csv_mode 면 "ms, 값 ,...", 아니면 " 이름 : 값 ,..."

제출 체크리스트

- 네 센서가 한 줄 · 플로터에 네 개의 선
- DHT 2 초 , 나머지는 더 자주
- m 으로 CSV 전환 , 머리줄 한 번
- 센서 추가 = 표에 한 줄

확장 : 링 버퍼 필터를 sensor_t 에 · alarm_t 로 30°C 경보 · CSV 를 엑셀 그래프로 (Day 4 에서는 파일로 저장)

오늘의 여섯 문장

- 1 ADC 는 12 비트 (0~4095). 전압이 필요하다면 `analogReadMilliVolts`, 핀은 ADC1(32~39)
- 2 저항 센서는 10k Ω 과 분압해서 읽고 , 흔들림은 링 버퍼 이동평균으로 줄인다
- 3 서보는 50Hz 펄스 폭 (0.5~2.5ms). 조이스틱은 3V3, 서보는 5V, GND 는 공통
- 4 5V 출력 (MQ-2 AO, HC-SR04 Echo) 은 반드시 분압 . LM35 전원은 5V, 출력은 그대로
- 5 경보는 히스테리시스로 떨어지지 않게 , 실패 (NaN · 타임아웃) 는 반드시 검사
- 6 선언은 .h, 정의는 .cpp, 공유 변수는 `extern`. 센서는 구조체 + 함수 포인터 표로

Day 3 예고 7-Segment · 74HC595 · 4 자리 멀티플렉싱 · 8×8 도트 매트릭스 · I2C(1602 LCD · PCF8591) · DC 모터 · 스테퍼 · 릴레이 .
오늘의 센서 값을 LCD 와 7-Segment 에 표시하고 모터를 움직입니다 .